

EXPERIMENT-17

Implement Mobile Price Prediction using a suitable Machine Learning algorithm.

Code:

```
import pandas as pd
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

print("=" * 70)
print("          MOBILE PRICE PREDICTION")
print("=" * 70)

# -----
# STEP 1: UPLOAD DATASET
# -----

uploaded = files.upload()
filename = list(uploaded.keys())[0]
data = pd.read_csv(filename)
print("\nDataset Loaded Successfully!")

# -----
# STEP 2: DISPLAY DATASET
# -----

print("\n" + "=" * 70)
print("          MOBILE DATASET")
print("=" * 70)
print(data.head(15))
```

```

print("\nTotal Number of Records :", len(data))

print("\nColumn Names:")

print(list(data.columns))

# -----

# STEP 3: FIND TARGET COLUMN

# -----

target = None

# Check common target column names

possible_targets = [
    "price_range",
    "Price_Range",
    "price range",
    "Price Range",
    "price",
    "Price",
    "label",
    "Label",
    "target",
    "Target",
    "category",
    "Category"
]

for col in possible_targets:
    if col in data.columns:
        target = col
        break

# If none of the common names exist,
# use the LAST COLUMN as target

if target is None:

```

```

    target = data.columns[-1]
print("\n" + "=" * 70)
print("          TARGET INFORMATION")
print("=" * 70)
print("\nTarget Column :", target)
print("\nTarget Values:")
print(data[target].value_counts())

# -----
# STEP 4: SELECT NUMERICAL FEATURES
# -----

X = data.drop(columns=[target])
# Keep only numerical columns
X = X.select_dtypes(include="number")
y = data[target]
print("\nNumber of Input Features :", X.shape[1])
print("\nInput Features:")
print(list(X.columns))

# -----
# STEP 5: TRAIN TEST SPLIT
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42
)
print("\n" + "=" * 70)
print("          DATA SPLITTING")

```

```

print("=" * 70)

print("\nTraining Records :", len(X_train))

print("Testing Records :", len(X_test))

# -----
# STEP 6: CREATE MODEL
# -----

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

print("\n" + "=" * 70)

print("          TRAINING RANDOM FOREST")

print("=" * 70)

model.fit(X_train, y_train)

print("\nModel Training Completed Successfully!")

# -----
# STEP 7: PREDICTION
# -----

prediction = model.predict(X_test)

print("\n" + "=" * 70)

print("          ACTUAL VS PREDICTED")

print("=" * 70)

# Display first 30 results
for i in range(min(30, len(y_test))):
    print(
        "Record", i + 1,
        " | Actual Price Category =",
        y_test.iloc[i],
    )

```

```

        " | Predicted Price Category =",
        prediction[i]
    )

# -----
# STEP 8: ACCURACY
# -----

accuracy = accuracy_score(y_test, prediction)
print("\n" + "=" * 70)
print("          MODEL PERFORMANCE")
print("=" * 70)
print("\nAccuracy :", round(accuracy * 100, 2), "%")

# -----
# STEP 9: CONFUSION MATRIX
# -----

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, prediction))

# -----
# STEP 10: FEATURE IMPORTANCE
# -----

print("\n" + "=" * 70)
print("          FEATURE IMPORTANCE")
print("=" * 70)

importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

importance = importance.sort_values(

```

```

        by="Importance",
        ascending=False
    )
    print(importance.to_string(index=False))

# -----
# FINAL RESULT
# -----

print("\n" + "=" * 70)
print("      MOBILE PRICE PREDICTION COMPLETED")
print("=" * 70)

print("\nTarget Column Used :", target)
print("Number of Features :", X.shape[1])
print("Training Samples  :", len(X_train))
print("Testing Samples   :", len(X_test))
print("Model Used        : Random Forest Classifier")
print("Accuracy           :", round(accuracy * 100, 2), "%")
print("\nFinal Result:")

print("The Random Forest algorithm successfully classified")
print("the mobile phones into their respective price categories.")
print("=" * 70)

```

Sample Input:

```

... =====
                        MOBILE PRICE PREDICTION
=====
Choose Files Mobile.csv
Mobile.csv(text/csv) - 252087 bytes, last modified: 8/10/2026 - 100% done
Saving Mobile.csv to Mobile.csv

```

Sample Output:

Dataset Loaded Successfully!

...

```
=====
MOBILE DATASET
=====
Battery_power_mAh Bluetooth Speed_of_microprocessor Dual_sim Front_camera \
0      842 mAh      No      2.2      No      1 pixels
1     1021 mAh     Yes      0.5     Yes      0 pixels
2      563 mAh     Yes      0.5     Yes      2 pixels
3      615 mAh     Yes      2.5     No      0 pixels
4     1821 mAh     Yes      1.2     No     13 pixels
5     1859 mAh     No      0.5     Yes      3 pixels
6     1821 mAh     No      1.7     No      4 pixels
7     1954 mAh     No      0.5     Yes      0 pixels
8     1445 mAh     Yes      0.5     No      0 pixels
9      509 mAh     Yes      0.6     Yes      2 pixels
10     769 mAh     Yes      2.9     Yes      0 pixels
11    1520 mAh     Yes      2.2     No      5 pixels
12    1815 mAh     No      2.8     No      2 pixels
13     803 mAh     Yes      2.1     No      7 pixels
14    1866 mAh     No      0.5     No     13 pixels
```

```
4G Internal_memeory_gb Mobile_depth Mobile_weight Cores_of_processor \
0      No           7 gb      0.6 cm      188 g           2
1     Yes          53 gb      0.7 cm      136 g           3
2     Yes          41 gb      0.9 cm      145 g           5
3      No          10 gb      0.8 cm      131 g           6
4     Yes          44 gb      0.6 cm      141 g           2
5      No          22 gb      0.7 cm      164 g           1
6     Yes          10 gb      0.8 cm      139 g           8
7      No          24 gb      0.8 cm      187 g           4
8      No          53 gb      0.7 cm      174 g           7
9     Yes           9 gb      0.1 cm       93 g           5
10     No           9 gb      0.1 cm      182 g           5
11     Yes          33 gb      0.5 cm      177 g           8
12     No          33 gb      0.6 cm      159 g           4
13     No          17 gb      1.0 cm      198 g           4
14     Yes          52 gb      0.7 cm      185 g           1
```

```
...
... px_height Pixel_width Ram_mb Screen_height Screen_weight talk_time \
0 ... 20 ppcm 756 ppcm 2549 mb 9 cm 7 cm 19
1 ... 905 ppcm 1988 ppcm 2631 mb 17 cm 3 cm 7
2 ... 1263 ppcm 1716 ppcm 2603 mb 11 cm 2 cm 9
3 ... 1216 ppcm 1786 ppcm 2769 mb 16 cm 8 cm 11
4 ... 1208 ppcm 1212 ppcm 1411 mb 8 cm 2 cm 15
5 ... 1004 ppcm 1654 ppcm 1067 mb 17 cm 1 cm 10
6 ... 381 ppcm 1018 ppcm 3220 mb 13 cm 8 cm 18
7 ... 512 ppcm 1149 ppcm 700 mb 16 cm 3 cm 5
8 ... 386 ppcm 836 ppcm 1099 mb 17 cm 1 cm 20
9 ... 1137 ppcm 1224 ppcm 513 mb 19 cm 10 cm 12
10 ... 248 ppcm 874 ppcm 3946 mb 5 cm 2 cm 7
11 ... 151 ppcm 1005 ppcm 3826 mb 14 cm 9 cm 13
12 ... 607 ppcm 748 ppcm 1482 mb 18 cm 0 cm 2
13 ... 344 ppcm 1440 ppcm 2680 mb 7 cm 1 cm 4
14 ... 356 ppcm 563 ppcm 373 mb 14 cm 9 cm 3
```

```
3G touch_screen wifi price_range
0 No No Yes Medium cost
1 Yes Yes No High cost
2 Yes Yes No High cost
3 Yes No No High cost
4 Yes Yes No Medium cost
5 Yes No No Medium cost
6 Yes No Yes Very High cost
7 Yes Yes Yes Low cost
8 Yes No No Low cost
9 Yes No No Low cost
10 No No No Very High cost
11 Yes Yes Yes Very High cost
12 Yes No No Medium cost
13 Yes No Yes High cost
14 Yes No Yes Low cost
```

[15 rows x 21 columns]

Total Number of Records : 2000

```

Column Names:
... ['Battery_power_mAh', 'BluetooH', 'Speed_of_microprocessor', 'Dual_sim', 'Front_c

=====
                        TARGET INFORMATION
=====

Target Column : price_range

Target Values:
price_range
Medium cost      500
High cost        500
Very High cost   500
Low cost         500
Name: count, dtype: int64

Number of Input Features : 3

Input Features:
['Speed_of_microprocessor', 'Cores_of_processor', 'talk_time']

=====
                        DATA SPLITTING
=====

Training Records : 1400
Testing Records  : 600

=====
                        TRAINING RANDOM FOREST
=====

Model Training Completed Successfully!

...
=====
                        ACTUAL VS PREDICTED
=====
Record 1 | Actual Price Category = Low cost | Predicted Price Category = Very High cost
Record 2 | Actual Price Category = High cost | Predicted Price Category = Very High cost
Record 3 | Actual Price Category = Medium cost | Predicted Price Category = High cost
Record 4 | Actual Price Category = Very High cost | Predicted Price Category = High cost
Record 5 | Actual Price Category = Medium cost | Predicted Price Category = Low cost
Record 6 | Actual Price Category = Medium cost | Predicted Price Category = High cost
Record 7 | Actual Price Category = High cost | Predicted Price Category = Very High cost
Record 8 | Actual Price Category = Low cost | Predicted Price Category = Low cost
Record 9 | Actual Price Category = Very High cost | Predicted Price Category = Very High cost
Record 10 | Actual Price Category = Medium cost | Predicted Price Category = Very High cost
Record 11 | Actual Price Category = Low cost | Predicted Price Category = High cost
Record 12 | Actual Price Category = Low cost | Predicted Price Category = Medium cost
Record 13 | Actual Price Category = High cost | Predicted Price Category = Medium cost
Record 14 | Actual Price Category = Very High cost | Predicted Price Category = Medium cost
Record 15 | Actual Price Category = Very High cost | Predicted Price Category = Low cost
Record 16 | Actual Price Category = High cost | Predicted Price Category = Very High cost
Record 17 | Actual Price Category = Very High cost | Predicted Price Category = Very High cost
Record 18 | Actual Price Category = Very High cost | Predicted Price Category = High cost
Record 19 | Actual Price Category = Medium cost | Predicted Price Category = High cost
Record 20 | Actual Price Category = Low cost | Predicted Price Category = Very High cost
Record 21 | Actual Price Category = Low cost | Predicted Price Category = Very High cost
Record 22 | Actual Price Category = High cost | Predicted Price Category = Medium cost
Record 23 | Actual Price Category = Medium cost | Predicted Price Category = Low cost
Record 24 | Actual Price Category = High cost | Predicted Price Category = Low cost
Record 25 | Actual Price Category = Low cost | Predicted Price Category = Medium cost
Record 26 | Actual Price Category = Medium cost | Predicted Price Category = Low cost
Record 27 | Actual Price Category = Very High cost | Predicted Price Category = Low cost
Record 28 | Actual Price Category = High cost | Predicted Price Category = Low cost
Record 29 | Actual Price Category = High cost | Predicted Price Category = Very High cost
Record 30 | Actual Price Category = Low cost | Predicted Price Category = High cost

```


...

```
=====
MODEL PERFORMANCE
=====
```

Accuracy : 25.0 %

Confusion Matrix:

```
[[34 37 34 43]
 [42 32 45 32]
 [33 38 38 37]
 [25 36 48 46]]
```

```
=====
FEATURE IMPORTANCE
=====
```

	Feature	Importance
	talk_time	0.409530
Speed_of_microprocessor		0.400538
Cores_of_processor		0.189931

```
=====
MOBILE PRICE PREDICTION COMPLETED
=====
```

Target Column Used : price_range
Number of Features : 3
Training Samples : 1400
Testing Samples : 600
Model Used : Random Forest Classifier
Accuracy : 25.0 %

Final Result:

The Random Forest algorithm successfully classified
the mobile phones into their respective price categories.